

Documento técnico

CRM Omnicanal sin IA integrada

MVP, arquitectura, base de datos, canales Meta y asignación de chats a vendedores

Enfoque del documento

La IA no será un módulo funcional dentro del CRM por ahora. La IA se usará solamente como herramienta externa para ayudar a escribir código, revisar arquitectura y generar módulos. El producto será un CRM con inbox omnicanal, reglas simples de bot, enrutamiento de conversaciones y gestión manual por vendedores.

Versión 1.0 · Junio 2026

Índice del documento

1. Decisión principal y alcance
2. Concepto del producto
3. MVP recomendado
4. Arquitectura técnica
5. Stack recomendado
6. Módulos del sistema
7. Bot simple y asignación a vendedores
8. Integraciones con WhatsApp, Instagram y Facebook
9. Modelo de base de datos
10. Estructura del repositorio
11. Flujos principales del sistema
12. API interna sugerida
13. Seguridad, permisos y operación
14. Roadmap por fases
15. Cómo usar IA para construir el código sin meter IA al CRM
16. Prompt maestro para construir el MVP
17. Checklist de desarrollo
18. Fuentes oficiales consultadas

1. Decisión principal y alcance

Decisión: el CRM se construirá sin inteligencia artificial integrada en el producto. La IA no responderá clientes, no calificará leads, no resumirá conversaciones y no tomará decisiones dentro del sistema en esta primera versión.

Uso permitido de IA: la IA se usará como asistente de desarrollo externo para generar código, revisar módulos, crear migraciones, escribir pruebas, documentar endpoints y acelerar tareas técnicas.

Idea correcta para esta etapa

Primero construir un CRM funcional, vendible y estable. Después, cuando el flujo comercial esté probado, se puede agregar IA como módulo opcional.

Dentro del alcance actual	Fuera del alcance actual
CRM multiempresa	IA conversacional integrada
Inbox omnicanal	Respuestas generadas por IA
WhatsApp, Instagram y Messenger/Facebook por APIs oficiales	Agentes autónomos que ejecuten acciones
Bot simple de bienvenida y enrutamiento	RAG, embeddings o base de conocimiento inteligente
Asignación de chats a vendedores	Scoring predictivo de leads
Embudo de ventas, contactos, tareas y notas	Automatizaciones complejas con modelos de lenguaje

2. Concepto del producto

El CRM debe ser un centro de atención y ventas para negocios que reciben clientes por redes sociales y mensajería. El sistema debe capturar mensajes, convertirlos en conversaciones, asociarlos a contactos y permitir que un vendedor gestione todo desde un panel.

Cliente escribe por WhatsApp / Instagram / Facebook



Webhook oficial de Meta recibe el mensaje



Backend normaliza el mensaje



Se crea o actualiza contacto + conversación



Bot simple saluda y clasifica por reglas



Sistema asigna el chat a un vendedor



Vendedor atiende desde el CRM



Lead avanza en el embudo comercial

- El valor principal no es la IA, sino centralizar conversaciones y no perder leads.
- El vendedor debe ver todo el historial del cliente, sin importar si escribió por WhatsApp, Instagram o Facebook.
- El sistema debe permitir trabajar por equipos: varios vendedores, supervisores y administradores.
- El negocio debe poder medir atención, tiempos de respuesta, ventas y estado de cada lead.

3. MVP recomendado

La primera versión debe ser pequeña, estable y vendible. La recomendación es empezar con WhatsApp y CRM básico. Instagram y Facebook se agregan después, cuando el flujo principal esté probado.

MVP 1 recomendado	Descripción
Login y organizaciones	Cada empresa tiene usuarios, roles y datos separados.
Contactos	Registro de clientes y prospectos con nombre, teléfono, redes, etiquetas y notas.
Leads y embudo	Pipeline comercial con etapas como Nuevo, Contactado, Interesado, Negociación, Ganado y Perdido.
Inbox WhatsApp	Recepción y envío de mensajes por WhatsApp Business Platform / Cloud API.
Bot simple	Saludo, menú básico, captura de intención y redirección a vendedor.
Asignación de chats	Round-robin, por disponibilidad, por vendedor específico o manual.
Panel del vendedor	Bandeja de conversaciones asignadas, datos del cliente, notas y cambio de etapa.
Panel admin	Usuarios, roles, configuración del canal, horarios y reglas básicas.
Auditoría básica	Registro de eventos importantes: asignaciones, cierres, cambios de estado y mensajes enviados.

Primera meta vendible

Un CRM con WhatsApp, contactos, leads, embudo y asignación de chats ya puede venderse a tiendas, agencias, negocios de servicios, ventas por Instagram, celulares, inmobiliarias, academias y comercios.

4. Arquitectura técnica

La arquitectura recomendada es un monolito modular. No conviene empezar con microservicios porque aumenta la complejidad, complica el despliegue y retrasa el MVP. Un monolito modular permite crecer sin perder orden.



- El backend debe recibir webhooks, validar eventos, normalizar mensajes y guardarlos.
- El frontend debe mostrar conversaciones en tiempo real al vendedor.
- Las tareas pesadas deben ir a colas: reintentos de envío, sincronización, procesamiento de adjuntos, cierre automático y notificaciones.
- Los canales externos deben estar encapsulados en adaptadores para que el CRM no dependa directamente de la estructura de cada API.

5. Stack recomendado

Área	Recomendación	Motivo
Frontend	Next.js + TypeScript	Buen ecosistema, SSR cuando haga falta, componentes modernos y facilidad para paneles SaaS.
Backend	NestJS + TypeScript	Arquitectura modular, inyección de dependencias, validaciones, testing y estructura empresarial.
Alternativa backend	Fastify + TypeScript	Más liviano y rápido, pero requiere definir más estructura manualmente.
Base de datos	PostgreSQL	Robusta, relacional, ideal para CRM, conversaciones, permisos y reportes.
ORM	Prisma	Productivo, claro, buen manejo de migraciones y tipado.
Colas	Redis + BullMQ	Jobs, reintentos, asignaciones, webhooks y tareas programadas.
Tiempo real	WebSockets o Server-Sent Events	Necesario para que el inbox se actualice sin refrescar la página.
Archivos	Cloudflare R2 / AWS S3 / MinIO	Guardar imágenes, audios, documentos y adjuntos de conversaciones.
Infraestructura inicial	Docker + VPS	Barato, controlable y suficiente para MVP.
Escalamiento futuro	Kubernetes o servicios administrados	Solo cuando haya varios clientes y carga real.

Stack recomendado final

TypeScript completo: Next.js en frontend, NestJS en backend, PostgreSQL como base principal, Prisma como ORM, Redis/BullMQ para colas, Docker desde el inicio y APIs oficiales de Meta para canales.

6. Módulos del sistema

Módulo	Responsabilidad
Auth	Login, sesiones, recuperación de contraseña, tokens y seguridad.
Organizations	Empresas, configuración general, planes y aislamiento de datos.
Users & Roles	Usuarios, vendedores, administradores, supervisores y permisos.
Contacts	Clientes, teléfonos, redes, etiquetas, notas y datos comerciales.
Leads & Pipelines	Embudo, etapas, oportunidades, estados y motivos de pérdida.
Conversations	Conversaciones por canal, estado, asignación y prioridad.
Messages	Mensajes entrantes y salientes, adjuntos, estados y errores.
Channels	Conexiones con WhatsApp, Instagram y Messenger/Facebook.
Routing Bot	Bot de bienvenida, menú simple y reglas de asignación.
Assignments	Asignación automática/manual a vendedores y transferencia de chats.
Tasks	Recordatorios, seguimientos y pendientes comerciales.
Reports	Métricas de atención, ventas, tiempos de respuesta y productividad.
Audit Logs	Registro de acciones importantes para seguridad y control.

7. Bot simple y asignación a vendedores

Sí, es totalmente correcto empezar con un bot pequeño. Ese bot no necesita IA. Puede funcionar con reglas, menús y estados simples. Su objetivo no es vender solo, sino ordenar la entrada de clientes y llevarlos rápido al vendedor correcto.

Función del bot	Ejemplo
Saludo inicial	Hola, gracias por escribir a [Empresa]. ¿En qué podemos ayudarte?
Menú básico	1) Comprar 2) Soporte 3) Crédito / cuotas 4) Hablar con asesor
Captura de datos	Nombre, producto de interés, ciudad o motivo de contacto.
Clasificación simple	Si elige comprar, va a ventas. Si elige soporte, va a soporte.
Asignación	El chat se asigna automáticamente a un vendedor disponible.
Notificación interna	El vendedor recibe el chat en su bandeja.
Transferencia	Supervisor o vendedor puede mover el chat a otra persona.

Estado inicial: NEW
└─ enviar saludo
└─ pedir opción

Estado: WAITING_OPTION

- └ opción 1: category = SALES
- └ opción 2: category = SUPPORT
- └ opción 3: category = FINANCING
- └ opción 4: category = HUMAN

Estado: ASSIGNING

- └ buscar vendedor disponible según reglas
- └ asignar conversación
- └ notificar al vendedor

Estado: ASSIGNED

- └ vendedor gestiona manualmente

Estado: CLOSED

- └ conversación resuelta o venta finalizada

Regla de asignación	Cuándo usarla
Round-robin	Todos los vendedores reciben chats de forma equilibrada. Buena opción inicial.
Por disponibilidad	Solo asigna a vendedores conectados o en horario.
Por especialidad	Ventas, soporte, financiamiento, garantías, repuestos, etc.
Por sucursal	Si el negocio tiene varias tiendas o ubicaciones.
Por cliente previo	Si el cliente ya tenía vendedor asignado, se conserva el mismo.
Manual	Un supervisor decide quién atiende el chat.

Recomendación práctica

Para el MVP usa round-robin + conservación del vendedor anterior. Eso es simple, justo para el equipo y útil para no romper la relación comercial con clientes recurrentes.

8. Integraciones con WhatsApp, Instagram y Facebook

Las integraciones deben hacerse por APIs oficiales de Meta. No se recomienda usar scraping, bots con navegador, sesiones no oficiales ni automatizaciones que simulen WhatsApp Web o Instagram manualmente.

Canal	Integración recomendada	Uso dentro del CRM
WhatsApp	WhatsApp Business Platform / Cloud API	Recibir mensajes, enviar respuestas, usar plantillas aprobadas cuando corresponda, recibir estados de mensajes.
Instagram	Instagram Messaging API	Recibir DMs de cuentas profesionales conectadas, responder desde el CRM y centralizar conversaciones.
Facebook	Messenger Platform para Facebook Pages	Recibir mensajes enviados a la página, responder desde el CRM y asignar conversaciones.

- Cada empresa dentro del CRM debe poder conectar sus propias cuentas Meta.
- El sistema debe guardar tokens y credenciales cifradas.
- Cada mensaje entrante debe ser recibido por webhook y normalizado a una estructura interna común.
- Los mensajes salientes deben pasar por un servicio de envío con reintentos y registro de errores.
- WhatsApp tiene costos y reglas de mensajería; el sistema debe estar preparado para distinguir mensajes de servicio, utilidad, autenticación y marketing.

- Para producción se necesitará Meta App, permisos correctos, revisión de aplicación cuando aplique, webhooks públicos HTTPS y configuración de negocios.

```
NormalizedMessage {
  organizationId: string
  channel: "whatsapp" | "instagram" | "messenger"
  externalConversationId: string
  externalMessageId: string
  senderExternalId: string
  contactExternalId: string
  direction: "inbound" | "outbound"
  type: "text" | "image" | "audio" | "video" | "document" | "interactive"
  text?: string
  attachments?: Attachment[]
  timestamp: Date
  rawPayload: Json
}
```

9. Modelo de base de datos

El modelo debe diseñarse como multiempresa desde el primer día. Esto evita reescribir el sistema cuando se quiera vender a varios negocios.

Tabla	Propósito
organizations	Empresas/clientes que usan el CRM.
users	Usuarios del sistema.
organization_users	Relación entre empresas y usuarios, con rol y estado.
roles	Roles como owner, admin, supervisor, seller.
permissions	Permisos granulares si se quiere crecer a RBAC avanzado.
contacts	Personas/clientes/prospectos.
contact_channels	WhatsApp, Instagram, Facebook u otros identificadores externos de un contacto.
leads	Oportunidades comerciales asociadas a contactos.
pipelines	Embudo comercial por empresa.
pipeline_stages	Etapas del embudo.
conversations	Conversaciones por canal y contacto.
messages	Mensajes entrantes y salientes.
message_attachments	Archivos asociados a mensajes.
message_statuses	Enviado, entregado, leído, fallido.
channel_connections	Conexiones Meta por empresa.
routing_rules	Reglas de enrutamiento y asignación.
assignments	Historial de asignaciones de conversaciones.
tasks	Seguimientos, llamadas y pendientes.
notes	Notas internas del equipo sobre contactos/leads.
audit_logs	Registro de acciones críticas.

```
organizations
  id
  name
  status
  timezone
  created_at

users
  id
  name
```

```
email
password_hash
status
created_at

organization_users
  organization_id
  user_id
  role
  is_active

contacts
  id
  organization_id
  full_name
  phone
  email
  tags
  created_at

contact_channels
  id
  contact_id
  channel
  external_id
  display_name
  username

conversations
  id
  organization_id
  contact_id
  channel_connection_id
  channel
  status
  assigned_user_id
  last_message_at
  created_at

messages
  id
  conversation_id
  direction
  channel
  external_message_id
  type
  text
  status
  sent_by_user_id
  created_at

leads
  id
  organization_id
  contact_id
  pipeline_id
  stage_id
  value
  currency
  status
```



```
assigned_user_id
created_at
```

10. Estructura del repositorio

Para un MVP rápido y ordenado, conviene usar un monorepo. Así frontend, backend y paquetes compartidos viven juntos.

```
crm-omnicanal/
  apps/
    web/           # Next.js frontend
    api/           # NestJS backend
  packages/
    shared/        # Tipos compartidos, DTOs y utilidades
    config/        # Configuración común
  infra/
    docker/
    nginx/
    postgres/
    redis/
  docs/
    PRD.md
    ARCHITECTURE.md
    DATABASE_SCHEMA.md
    API_CONTRACT.md
    ROADMAP.md
  scripts/
    seed.ts
    backup.sh
  docker-compose.yml
  README.md

apps/api/src/
  modules/
    auth/
    organizations/
    users/
    contacts/
    leads/
    pipelines/
    conversations/
    messages/
    channels/
      whatsapp/
      instagram/
      messenger/
    adapters/
    routing-bot/
    assignments/
    tasks/
    reports/
    audit/
  common/
    database/
    queue/
    storage/
    config/
```

```
guards/
decorators/
```

11. Flujos principales del sistema

11.1 Flujo de mensaje entrante

1. Cliente envía mensaje por WhatsApp, Instagram o Facebook.
2. Meta envía un webhook al backend.
3. El backend valida firma, token o configuración del webhook.
4. Se guarda el payload crudo para auditoría y debugging.
5. El adaptador del canal convierte el evento a NormalizedMessage.
6. El sistema busca o crea el contacto.
7. El sistema busca o crea la conversación.
8. Se guarda el mensaje.
9. El bot simple evalúa si debe saludar, pedir opción o asignar.
10. Se notifica al frontend en tiempo real.
11. El vendedor ve la conversación en su bandeja.

11.2 Flujo de mensaje saliente

1. Vendedor escribe desde el CRM.
2. Backend valida permisos y estado de la conversación.
3. Se crea registro de mensaje saliente en estado pending.
4. Se envía job a la cola.
5. El adaptador del canal llama la API oficial correspondiente.
6. Se actualiza estado: sent, delivered, read o failed según webhooks posteriores.
7. El frontend actualiza la conversación en tiempo real.

11.3 Flujo de asignación

1. Nueva conversación entra sin vendedor asignado.
2. Sistema revisa si el contacto ya tenía vendedor anterior.
3. Si existe y está activo, se asigna al mismo vendedor.
4. Si no existe, se aplica round-robin entre vendedores disponibles.
5. Se registra la asignación en assignments.
6. Se notifica al vendedor.
7. Supervisor puede reasignar manualmente si hace falta.

12. API interna sugerida

Método	Endpoint	Propósito
POST	/auth/login	Iniciar sesión.
GET	/me	Obtener usuario actual y empresa activa.
GET	/organizations/:id/users	Listar usuarios de una empresa.
POST	/contacts	Crear contacto.
GET	/contacts	Buscar/listar contactos.
GET	/contacts/:id	Ver contacto e historial.

Método	Endpoint	Propósito
POST	/leads	Crear oportunidad comercial.
PATCH	/leads/:id/stage	Mover lead de etapa.
GET	/conversations	Listar conversaciones por estado/asignación.
GET	/conversations/:id/messages	Ver mensajes de una conversación.
POST	/conversations/:id/messages	Enviar mensaje desde el CRM.
PATCH	/conversations/:id/assign	Asignar o reasignar chat.
PATCH	/conversations/:id/close	Cerrar conversación.
POST	/webhooks/meta/whatsapp	Webhook WhatsApp.
POST	/webhooks/meta/messenger	Webhook Messenger/Facebook.
POST	/webhooks/meta/instagram	Webhook Instagram.
GET	/reports/sales	Métricas comerciales.
GET	/reports/attention	Métricas de atención.

13. Seguridad, permisos y operación

- Aislar datos por organization_id en todas las consultas.
- Usar RBAC desde el inicio: owner, admin, supervisor, seller.
- Cifrar tokens de Meta y secretos de canales.
- Registrar audit logs para cambios de permisos, conexiones, asignaciones y cierre de chats.
- Validar webhooks y evitar aceptar eventos de fuentes no verificadas.
- Implementar rate limiting en endpoints públicos.
- Guardar payloads crudos de webhooks por tiempo limitado para debugging.
- Aplicar backups automáticos diarios de PostgreSQL.
- Separar entornos: development, staging y production.
- Usar HTTPS obligatorio en producción.

Rol	Permisos recomendados
Owner	Todo: facturación, configuración, usuarios, canales y datos.
Admin	Configurar usuarios, canales, pipelines y reportes.
Supervisor	Ver equipo, reasignar chats, revisar conversaciones y reportes.
Seller	Ver sus chats, responder, crear notas, mover leads y cerrar conversaciones.

14. Roadmap por fases

Fase	Objetivo	Resultado
Fase 0	Definición técnica	PRD, arquitectura, esquema de base de datos y diseño del MVP.
Fase 1	Base SaaS	Login, empresas, usuarios, roles y layout principal.
Fase 2	CRM básico	Contactos, leads, pipelines, notas y tareas.
Fase 3	Inbox WhatsApp	Webhook, recepción, envío, conversación y estados.
Fase 4	Bot y asignaciones	Saludo, menú, round-robin y bandeja del vendedor.
Fase 5	Reportes básicos	Tiempos de respuesta, cantidad de chats, ventas y productividad.
Fase 6	Instagram y Messenger	Integrar DMs de Instagram y mensajes de Facebook Page.

Fase	Objetivo	Resultado
Fase 7	Automatizaciones simples	Horarios, respuestas rápidas, recordatorios y reglas.
Fase futura	IA opcional	Solo si el CRM ya funciona bien y hay casos de uso claros.

15. Cómo usar IA para construir el código sin meter IA al CRM

La IA puede ser muy útil como herramienta de desarrollo, pero debe recibir tareas pequeñas y precisas. No se debe pedir: "construye un CRM completo". Eso produce código desordenado. Hay que pedir módulos aislados con criterios claros.

Forma incorrecta	Forma correcta
Hazme un CRM con WhatsApp, Instagram y Facebook.	Construye el módulo Contacts con NestJS, Prisma y PostgreSQL. Debe ser multiempresa y tener CRUD, validaciones, permisos y pruebas.
Crea todo el backend.	Crea primero Auth + Organizations + Users. No toques conversaciones ni canales todavía.
Hazme la integración con Meta.	Crea un adaptador WhatsApp que reciba un webhook, normalice mensajes y los guarde en la tabla messages.
Agrega el bot.	Crea una máquina de estados para routing-bot: NEW, WAITING_OPTION, ASSIGNING, ASSIGNED, CLOSED.

- Mantén archivos de documentación dentro de /docs para que la IA tenga contexto fijo.
- Pide un módulo por vez.
- Exige tests básicos y migraciones.
- Revisa manualmente seguridad, permisos, multiempresa y manejo de errores.
- No permitas que la IA cambie arquitectura sin autorización.

16. Prompt maestro para construir el MVP

Este prompt puede usarse como base con una IA programadora. Conviene pegarlo al inicio del proyecto y luego pedir tareas por módulos.

Actúa como arquitecto y desarrollador senior full-stack.

Vamos a construir un CRM omnicanal SIN inteligencia artificial integrada en el producto.
La IA solo se usará como herramienta externa para ayudarte a generar código. El CRM no debe generar respuestas con IA, no debe calificar leads con IA y no debe tomar decisiones inteligentes.

Objetivo del producto:

Crear un CRM SaaS multiempresa con inbox de WhatsApp, Instagram y Facebook Messenger, contactos, leads, embudo comercial, tareas, notas, bot simple por reglas y asignación de chats a vendedores.

Stack obligatorio:

- Frontend: Next.js + TypeScript
- Backend: NestJS + TypeScript
- Base de datos: PostgreSQL
- ORM: Prisma
- Colas: Redis + BullMQ
- Tiempo real: WebSockets o Server-Sent Events
- Infraestructura: Docker Compose para desarrollo local

Arquitectura:

- Monolito modular, no microservicios.
- Multiempresa desde el primer día usando `organization_id`.
- RBAC básico: owner, admin, supervisor, seller.
- Integraciones externas encapsuladas en adapters.
- Todo canal externo debe convertirse a un `NormalizedMessage` interno.

MVP inicial:

1. Auth
2. Organizations
3. Users/Roles
4. Contacts
5. Leads/Pipelines
6. Conversations/Messages
7. WhatsApp webhook + envío de mensajes
8. Routing bot simple
9. Assignment de conversaciones a vendedores
10. Panel básico de vendedor y admin

Reglas:

- No construyas todo de una vez.
- Trabaja módulo por módulo.
- Cada módulo debe incluir DTOs, servicios, controladores, validaciones, manejo de errores y pruebas básicas.
- Cada consulta debe respetar `organization_id`.
- No mezcles lógica de canales con lógica de CRM; usa adapters.
- No agregues IA al producto.
- Antes de escribir código, explica brevemente la estructura que vas a crear.

Primera tarea:

Genera la estructura inicial del monorepo y el backend base con Auth, Organizations y Users.

17. Checklist de desarrollo

Área	Checklist
Producto	Definir nicho inicial, flujo de venta, etapas de pipeline y roles del equipo.
Diseño	Crear pantallas: login, dashboard, contactos, leads, inbox, conversación, configuración.
Backend	Configurar NestJS, Prisma, PostgreSQL, Redis, validaciones y módulos base.
Frontend	Layout SaaS, navegación, tablas, formularios e inbox.
Base de datos	Migraciones iniciales y seed de roles/empresa demo.
WhatsApp	Meta App, número, webhook, token, envío y recepción.
Bot	Estados simples, menú, horarios, reglas y fallback a humano.
Asignación	Round-robin, vendedor anterior, manual y reasignación.
Seguridad	RBAC, organización en todas las consultas, cifrado de tokens y audit logs.
Operación	Docker Compose, backups, logs, variables de entorno y despliegue VPS.
Pruebas	Pruebas unitarias de servicios y pruebas de flujo webhook/conversación.

18. Fuentes oficiales consultadas

Estas fuentes se deben revisar directamente durante la implementación porque Meta puede cambiar requisitos, permisos, tarifas o políticas.

- Meta for Developers — WhatsApp Business Platform / Cloud API:
<https://developers.facebook.com/documentation/business-messaging/whatsapp/about-the-platform>
- Meta for Developers — WhatsApp Webhooks:
<https://developers.facebook.com/documentation/business-messaging/whatsapp/webhooks/overview/>
- Meta for Developers — WhatsApp messages webhook reference:
<https://developers.facebook.com/documentation/business-messaging/whatsapp/webhooks/reference/messages>
- WhatsApp Business — Business Platform Pricing: <https://whatsappbusiness.com/products/platform-pricing/>
- Meta for Developers — Messenger Platform Overview:
<https://developers.facebook.com/documentation/business-messaging/messenger-platform/overview>
- Meta for Developers — Messenger Platform Webhooks:
<https://developers.facebook.com/documentation/business-messaging/messenger-platform/webhooks>
- Meta for Developers — Instagram Messaging Overview:
<https://developers.facebook.com/documentation/business-messaging/instagram-messaging/overview>
- Meta for Developers — Instagram Messaging Webhooks:
<https://developers.facebook.com/documentation/business-messaging/instagram-messaging/webhooks>

Cierre recomendado

La mejor ruta es construir primero un CRM funcional con WhatsApp e inbox, usando bot simple y asignación a vendedores. Eso reduce complejidad y permite validar si el producto se vende. Luego se agregan Instagram, Messenger, reportes avanzados y automatizaciones. La IA puede quedar para una fase futura cuando ya existan datos reales y procesos comerciales claros.